

Projet d'Intelligence Artificielle

Génération d'I.A. appliquée à la musique

RÉALISÉ PAR

Duc Hoang Viet NGUYEN
duc-hoang-viet.nguyen.etu@univ-lille.fr

Elisa CASTANO
elisa.castano.etu@univ-lille.fr

Yvan Michel TCHONANG
yvan-michel.tchomang.etu@univ-lille.fr

Parcours Ingénierie Statistique et Numérique, Data Sciences (ISN)

18 Mai 2026

Table des matières

1	Introduction	1
2	Représentation des Données — Tokenisation MIDI	1
2.1	Principe	1
2.2	Quantisation	1
2.3	Structure d’une séquence	1
3	Prétraitement des fichiers MIDI	2
4	Tokenisation et construction du vocabulaire	2
5	Architecture du modèle	2
5.1	Vue d’ensemble	2
5.2	Composants clés	2
5.3	Génération autorégressive	3
6	Entraînement du modèle	3
6.1	Configuration	3
6.2	Loss pondérée	3
6.3	Diagnostic en cours d’entraînement	3
7	Modèle utilisé	4
8	Génération d’une nouvelle séquence musicale	4
9	Résultats obtenus	4
10	Conclusion	5

1 Introduction

L’objectif de ce projet est de créer une nouvelle musique à partir d’une séquence de notes donnée. Pour cela, nous construisons une chaîne complète allant du fichier MIDI initial jusqu’à la génération d’un nouveau fichier MIDI. Les notes sont d’abord transformées en tokens, puis ces tokens sont convertis en identifiants numériques afin d’être utilisés par le modèle. Le modèle apprend ainsi à prédire le token suivant à partir du contexte précédent. Après l’entraînement, il suffit de lui fournir quelques notes en entrée pour qu’il génère automatiquement une longue séquence musicale, qui est ensuite reconvertie en fichier MIDI jouable.

Le corpus d’entraînement est constitué des fichiers MIDI fournis dans le cadre du projet (GrandMidiPiano, accessible via la plateforme FileSender Renater). Le principe d’utilisation est simple : on fournit au modèle quelques notes en entrée (seed), et on observe la séquence musicale longue qu’il génère de manière autonome.

2 Représentation des Données — Tokenisation MIDI

Les données utilisées proviennent de fichiers MIDI de musique classique. Afin de réduire l’hétérogénéité des styles musicaux et de faciliter l’apprentissage du modèle, nous avons limité le dataset principalement aux œuvres de Mozart et Beethoven. Ce choix permet de conserver une certaine diversité musicale tout en gardant des structures harmoniques relativement cohérentes pour l’entraînement.

Pipeline général

MIDI $\rightarrow$ tokens $\rightarrow$ modèle Transformer $\rightarrow$ génération $\rightarrow$ MIDI

2.1 Principe

Les fichiers MIDI ne contiennent pas directement du son, mais une suite d’événements musicaux tels que les notes, leurs positions et leurs durées. Dans ce projet, chaque événement musical est transformé en une suite de tokens afin d’être utilisé par le modèle Transformer. Chaque note musicale est représentée par un quadruplet de quatre tokens consécutifs :

BAR $\rightarrow$ POSITION_X $\rightarrow$ VELOCITY_V $\rightarrow$ NOTE_ON_P $\rightarrow$ DURATION_D

- **POSITION** représente le moment où la note commence dans la mesure.
- **VELOCITY** correspond à l’intensité (volume) de la note.
- **NOTE_ON** correspond à la hauteur de la note MIDI (*pitch*).
- **DURATION** indique la durée de la note.

2.2 Quantisation

Afin de limiter la taille du vocabulaire et d’améliorer la robustesse du modèle, les valeurs brutes sont quantifiées.

- **Position** : grille de 120 ticks (TPQ = 480, GRID = TPQ // 4), soit une double-croche.
- **Durée** : 9 bins musicalement significatifs (de la croche à la double ronde).
- **Velocity** : 8 niveaux discrets : [16, 32, 48, 64, 80, 96, 112, 127].

Cette quantification réduit le vocabulaire à seulement **93 tokens** tout en préservant l’essentiel de l’expressivité musicale.

2.3 Structure d’une séquence

```
BOS   BAR   POSITION_0   VELOCITY_80   NOTE_ON_60   DURATION_480
      POSITION_480   VELOCITY_64   NOTE_ON_64   DURATION_240
      BAR   POSITION_0   VELOCITY_96   NOTE_ON_67   DURATION_960
      ...
      EOS
```

Les tokens spéciaux **BOS** (*Beginning of Sequence*) et **EOS** (*End of Sequence*) délimitent chaque pièce musicale.

3 Prétraitement des fichiers MIDI

Le prétraitement a été réalisé avec le fichier `filter.py`. Ce script parcourt le dossier contenant les fichiers MIDI originaux et sélectionne les morceaux dont le nom contient `Mozart` ou `Beethoven`, qu'il copie ensuite dans les dossiers `mozart_dataset` et `beethoven_dataset`.

Dans la suite du projet, nous avons principalement utilisé `mozart_dataset` pour construire le vocabulaire musical et entraîner le modèle.

4 Tokenisation et construction du vocabulaire

Après le prétraitement, les fichiers MIDI sélectionnés sont transformés en séquences de tokens avec le script `h0.py`. Comme expliqué précédemment, ces tokens représentent les informations principales des notes : leur position, leur hauteur, leur durée et leur intensité.

Le script ajoute également des tokens spéciaux comme `BOS`, `BAR` et `EOS`, qui permettent d'indiquer le début de la séquence, le changement de mesure et la fin de la séquence. À partir de l'ensemble des tokens obtenus, un vocabulaire est construit. Chaque token est ensuite associé à un identifiant numérique afin de pouvoir être utilisé par le modèle. Cette étape produit deux fichiers :

`vocab.txt` et `dataset.txt`

Le fichier `vocab.txt` contient l'ensemble des tokens possibles, tandis que `dataset.txt` contient la séquence musicale encodée sous forme numérique.

5 Architecture du modèle

5.1 Vue d'ensemble

Le modèle implémenté est un *decoder-only Transformer*, directement inspiré de GPT. Il prend en entrée une séquence de tokens et prédit le token suivant à chaque position (tâche de modélisation causale du langage).

Hyperparamètres retenus :

Paramètre	Valeur
Dimension d'embedding (<code>d_model</code>)	192
Nombre de têtes d'attention (<code>n_heads</code>)	6
Nombre de blocs Transformer (<code>n_layers</code>)	4
Dimension feed-forward (<code>d_ff</code>)	768
Longueur de contexte (<code>max_seq_len</code>)	256
Dropout	0.1
Total paramètres	~ 1,1M

Une amélioration possible consisterait à augmenter ces hyperparamètres (`d_model=256`, `n_heads=8`, `n_layers=6`, `max_seq_len=512`, ~ 1,8M paramètres), ce qui permettrait au modèle d'apprendre des structures musicales plus complexes et de capturer des dépendances à plus longue portée.

5.2 Composants clés

- **Multi-Head Self-Attention causale** : chaque position ne peut accéder qu'aux positions précédentes grâce à un masque triangulaire inférieur. Cela garantit le caractère autorégressif du modèle lors de la génération.
- **Feed-Forward avec GELU** : chaque bloc Transformer inclut un réseau feed-forward à deux couches avec activation GELU, appliqué indépendamment sur chaque position.
- **Pre-LayerNorm** : la normalisation est appliquée avant l'attention et le feed-forward (*architecture Pre-LN*), ce qui stabilise l'entraînement.

- **Weight Tying** : la matrice d’embedding des tokens et la couche de projection finale (*head*) partagent les mêmes poids, réduisant le nombre de paramètres et améliorant généralement la qualité des représentations.

5.3 Génération autorégressive

Lors de l’inférence, le modèle génère token par token en combinant deux techniques de filtrage :

- **Top-k sampling** : seuls les $k = 50$ tokens les plus probables sont conservés.
- **Top-p (nucleus) sampling** : on conserve le plus petit ensemble de tokens dont la probabilité cumulée dépasse $p = 0.85$.

Cette combinaison permet d’équilibrer cohérence et diversité dans les sorties générées. Le modèle utilisé est un Transformer auto-régressif inspiré des architectures GPT.

6 Entraînement du modèle

6.1 Configuration

Paramètre	Valeur
Optimiseur	AdamW ($\beta_1 = 0.9$, $\beta_2 = 0.95$, weight decay = 0.1)
Learning rate initial	3×10^{-4}
Scheduler	Cosine avec warmup (500 steps)
Batch size	32
Epochs max	50
Gradient clipping	1.0
Early stopping patience	12 epochs
Objectif (loss cible)	< 1.5

6.2 Loss pondérée

Un mécanisme de *weighted cross-entropy* est employé pour forcer le modèle à mieux apprendre les tokens musicalement importants, qui sont moins fréquents que les tokens structurels :

Type de token	Poids
NOTE_ON	$\times 2.0$
DURATION	$\times 2.0$
POSITION	$\times 2.5$
VELOCITY	$\times 1.0$ (neutre)
Autres (BAR, BOS, EOS)	$\times 1.0$

6.3 Diagnostic en cours d’entraînement

L’entraînement du modèle est réalisé avec le script `h2.py`. Ce programme charge le vocabulaire depuis `vocab.txt`, puis crée les séquences d’apprentissage à partir des fichiers MIDI de `mozart_dataset`. À chaque itération, le modèle reçoit une séquence de tokens en entrée et doit prédire le token suivant — la cible est donc la même séquence décalée d’un token.

La fonction de perte utilisée est la *CrossEntropyLoss* pondérée, et l’optimiseur est *AdamW*. Pour limiter le coût d’entraînement sur CPU/MPS, le programme sélectionne aléatoirement un sous-ensemble du dataset à chaque époque, et sauvegarde automatiquement le meilleur modèle obtenu.

À chaque epoch, une sonde diagnostique mesure la probabilité que le modèle prédise correctement un token NOTE_ON après avoir vu un token VELOCITY, afin de vérifier que le modèle apprend bien la grammaire musicale (POSITION $\rightarrow$ VELOCITY $\rightarrow$ NOTE_ON $\rightarrow$ DURATION).

Dans notre exécution, l’entraînement s’est terminé après 2 époques avec une perte finale d’environ 1.48, ayant atteint l’objectif cible (loss < 1.5). Le meilleur modèle, sélectionné selon la perte de validation, est

automatiquement sauvegardé dans `checkpoints/best_model.pt` et utilisé pour la génération de nouvelles séquences musicales.

7 Modèle utilisé

Le modèle utilisé dans ce projet est un modèle de type *Transformer*. Ce choix est adapté à notre objectif, car la musique peut être représentée comme une séquence d'événements, de manière similaire à une phrase composée de mots ou de tokens.

Dans notre cas, le modèle reçoit une suite de tokens musicaux et apprend à prédire le token suivant. Il fonctionne donc de manière auto-régressive : chaque nouveau token généré dépend des tokens précédents.

Ce type de modèle permet d'apprendre certaines régularités présentes dans les fichiers MIDI, comme l'enchaînement des notes, les durées, les positions dans la mesure et les intensités. Après l'entraînement, le modèle peut être utilisé pour générer une nouvelle séquence musicale à partir d'une courte séquence initiale.

Le modèle reste cependant relativement simple par rapport à des systèmes de génération musicale plus avancés. Il constitue une première approche permettant de construire une chaîne complète : transformation des fichiers MIDI en tokens, apprentissage des séquences musicales, puis génération d'un nouveau fichier MIDI.

8 Génération d'une nouvelle séquence musicale

La génération musicale est réalisée avec le script `h3.py`. Ce programme utilise le vocabulaire construit précédemment, contenu dans le fichier `vocab.txt`, ainsi que le modèle entraîné sauvegardé dans le fichier `best_model.pt`.

La génération commence à partir d'une courte séquence initiale, appelée *seed*. Cette séquence sert de point de départ au modèle. À partir de ces premiers tokens, le modèle prédit progressivement le token suivant, puis ajoute ce token à la séquence générée.

Le processus est donc auto-régressif : chaque nouveau token est généré en fonction des tokens précédents. Pour rendre la génération moins déterministe, le programme utilise une température et une sélection parmi les tokens les plus probables avec la méthode `top_k`. Cela permet d'obtenir des résultats plus variés.

Une fois la séquence de tokens générée, le programme la reconvertit en fichier MIDI via la bibliothèque `mido`. Les tokens comme `POSITION`, `VELOCITY`, `NOTE_ON` et `DURATION` sont utilisés pour reconstruire les notes, leurs positions, leurs intensités et leurs durées. Les morceaux générés sont ensuite sauvegardés dans le dossier `generated_music`. Dans notre cas, le programme génère plusieurs fichiers MIDI, par exemple :

```
song_1.mid  
song_2.mid  
song_3.mid
```

Ces fichiers peuvent ensuite être convertis en format audio `.wav` afin d'être écoutés. Ils peuvent également être importés sur Signal MIDI ou MIDI Toolbox.

Un point critique résolu en cours de projet : la bibliothèque `miditoolkit` utilisée initialement produisait des fichiers MIDI dont le header n'était pas reconnu par plusieurs lecteurs. Le remplacement par `mido` a permis de résoudre ce problème.

9 Résultats obtenus

L'entraînement s'est terminé après 2 époques avec une perte finale d'environ 1.48. Le modèle génère 5 morceaux de plusieurs dizaines de mesures, avec environ 1 247 notes par morceau. Nous avons principalement utilisé le dataset **Mozart** pour l'entraînement ; des tests ont également été effectués avec le dataset **Beethoven**, mais les résultats générés ne présentaient pas de différences musicales notables entre les deux corpus.

Du point de vue technique, toutes les étapes du projet ont été réalisées : filtrage des fichiers MIDI, tokenisation, construction du vocabulaire, entraînement, génération et conversion en fichier MIDI jouable.

À l’écoute, cependant, les résultats restent musicalement limités. Certaines séquences présentent des passages peu structurés ou des enchaînements de notes qui ne ressemblent pas toujours à une phrase musicale cohérente. Plusieurs facteurs expliquent ces limites :

- **Représentation simple** : les notes sont décrites uniquement par leur position, hauteur, durée et intensité, sans information sur la tonalité, les accords ou la structure globale du morceau.
- **Polyphonie excessive** : rien dans la représentation ne contraint le nombre de notes simultanées, ce qui produit parfois des passages trop denses.
- **Contexte limité** : avec `max_seq_len = 256`, le modèle ne peut mémoriser qu’une cinquantaine de notes — insuffisant pour des structures musicales à longue portée.
- **Apprentissage statistique** : le modèle apprend des régularités dans les données sans compréhension réelle de la mélodie ou de l’harmonie.

Pour améliorer ces résultats, plusieurs pistes sont envisageables : augmenter le contexte (`max_seq_len: 256` → `512`), ajouter un token de tonalité pour guider l’harmonie, contraindre la polyphonie maximale lors du décodage, ou utiliser un corpus plus large combinant plusieurs compositeurs. Cette dernière option permettrait d’améliorer la qualité musicale, mais nécessiterait une machine plus puissante, car l’entraînement sur un dataset plus important devient rapidement coûteux en temps et en mémoire.

10 Conclusion

Les résultats obtenus montrent que cette approche permet bien de générer une séquence musicale. Cependant, la qualité musicale reste limitée. Le modèle apprend principalement des régularités statistiques dans les données, sans comprendre réellement des notions musicales comme la tonalité, les accords, la mesure ou la structure d’une œuvre. Cela peut conduire à des résultats répétitifs, désorganisés ou sans style clairement identifiable.

Ce travail montre donc que la génération musicale ne dépend pas seulement de la quantité de données disponibles, mais aussi de la qualité de leur représentation, du choix du modèle et des connaissances musicales intégrées dans le code. Avec un code plus complexe, un modèle plus adapté et une meilleure prise en compte des éléments musicaux, il serait possible d’obtenir des séquences plus cohérentes et plus proches de véritables œuvres pour piano.

Enfin, ce projet met aussi en évidence une limite pratique : l’entraînement de modèles plus avancés ou sur une base de données plus importante demande davantage de ressources informatiques. Il constitue donc une première étape expérimentale, qui pourrait être améliorée par un meilleur prétraitement des données, une architecture plus puissante et une analyse musicale plus approfondie.

Références

- [1] R. Yuan et al., *ChatMusician : Understanding and Generating Music Intrinsically with LLMs* (HKUST, 2024).
Disponible sur : <https://arxiv.org/abs/2402.16153>
- [2] S.-L. Wu, Y. Kim, C.-Z. A. Huang, *MIDI-LLM : Adapting Large Language Models for Text-to-MIDI Music Generation* (NeurIPS 2025).
Disponible sur : <https://openreview.net/pdf?id=GVW9YixIAI>
- [3] A. Vaswani et al., *Attention Is All You Need*, NeurIPS (2017).
Disponible sur : <https://papers.nips.cc/paper/7181-attention-is-all-you-need.pdf>
- [4] C.-Z. A. Huang et al., *Music Transformer*, ICLR (2019).
Disponible sur : <https://arxiv.org/abs/1809.04281>